

Preparação para a apresentação de ESRT Fase B

1. COMANDOS CUPCARBON - SENSSCRIPT

1.1. Comandos Fundamentais de Comunicação

Qual é o comando que cria a trama?

Comando: data

Sintaxe: data <nome_variavel> <campo1> <campo2> <campo3> ...

Exemplos práticos do projeto:

1. Sensores de Estacionamento (PARK):

data p timestamp frame my_id GW_ID s

- timestamp: momento da detecção
- frame: tipo de mensagem ("PARK")
- my_id: ID do sensor que deteta ocupação
- GW_ID: ID do destino (Gateway)
- s: estado do sensor (0=livre, 1=ocupado)

2. Sensores Ambientais (DATA):

data p timestamp frame_type my_id GW_ID temp hum co2 pm10

- timestamp: momento da leitura
- frame_type: tipo de mensagem ("DATA")
- my_id: ID do aglomerado sensor
- GW_ID: ID do Gateway destino
- temp, hum, co2, pm10: valores dos sensores ambientais

3. Mensagem de Configuração Downlink:

data p_config timestamp frame my_id 15000

- Enviada pelo Gateway para alterar período de amostragem

Função: Cria uma mensagem formatada com múltiplos campos de dados que será enviada pela rede.

Qual é o comando que extrai os campos da trama?

Comando: vget

Sintaxe: vget <variavel_destino> <vetor> <indice>

Pré-requisito: Primeiro converter a mensagem recebida em vetor com vdata:

vdata v msg

Exemplos práticos do projeto:

1. No Gateway - Extrair tipo de mensagem:

receive msg 1000

vdata v msg

vget tipo v 1 # Extrai campo 1 (tipo: "PARK" ou "DATA")

2. No Gateway - Extrair dados de estacionamento:

if (tipo == "PARK")

vget sensor_id v 2 # ID do sensor

vget status v 4 # Estado (0=livre, 1=ocupado)

end

3. No Gateway - Extrair dados ambientais:

if (tipo == "DATA")

vget id_sensor v 2 # ID do aglomerado

vget temp v 4 # Temperatura

vget hum v 5 # Humidade

vget co2 v 6 # CO2

vget pm10 v 7 # PM10

end

4. Nos Aglomerados - Processar comando downlink:

receive msg_downlink sample_period

if (msg_downlink != "")

rdata msg_downlink cmd_tipo cmd_valor

if (cmd_tipo == "CONFIG")

set sample_period cmd_valor # Atualiza período de amostragem

end

end

Função: Extrai valores específicos de um vetor de dados previamente convertido.

Qual é o comando que lê o ID do nó?

Comando: `atget`

Sintaxe: `atget id <nome_variavel>`

Exemplo prático:

```
atget id my_id
```

```
cprint "O meu ID é:" my_id
```

Usado em todos os scripts do projeto:

- **Gateway.csc:** `atget id my_id` → ID 30
- **Sensores_Fila1.csc:** `atget id my_id` → IDs 21, 18, 19, 20, 22
- **Aglomerado_1.csc:** `atget id my_id` → ID 27
- **Aglomerado_2.csc:** `atget id my_id` → ID 33

Função: Obtém automaticamente o ID único do nó atribuído no CupCarbon e armazena numa variável para usar nas comunicações.

1.2. Leitura de Sensores

Sensores Digitais (Estacionamento)

Comando: `dreadsensor`

Sintaxe: `dreadsensor <variavel>`

Exemplo:

```
dreadsensor s
```

```
mark s          # Atualiza marcador visual (LED)
```

```
if (s != status)
```

```
    set status s
```

```
    # Envia mensagem PARK
```

```
end
```

Valores retornados:

- 0 = Lugar livre (sem obstáculo)
- 1 = Lugar ocupado (obstáculo detetado)

Usado em: Sensores_Fila1.csc, Sensores_Fila2.csc, Sensores_Fila3.csc, Sensores_Fila4.csc

Sensores Analógicos (Ambientais)

Comando: areadsensor

Sintaxe: *areadsensor* <variavel>

Exemplo:

```
areadsensor s
```

```
if (s != "X")
  vdata values s
  vget temp values 2    # Extrai temperatura
  vget hum values 4     # Extrai humidade
  vget co2 values 6     # Extrai CO2
  vget pm10 values 8    # Extrai PM10

  # Processa e envia dados
end
```

Retorna: String com múltiplos valores lidos de ficheiros .evt (temperatura, humidade, CO2, PM10)

Usado em: Aglomerado_1.csc, Aglomerado_2.csc

1.3. Comandos de Potência de Transmissão

Como alterar o PL (Power Level) em SenScript de forma a adaptar-se à distância?

Comando: atpl

Sintaxe: atpl <valor>

Valores típicos: 0 a 255 (quanto maior, maior o alcance)

Exemplos do projeto:

1. **Sensores de Estacionamento (curta distância - 3m entre sensores, 8m até BS):** atpl 20
 - Baixa potência porque estão próximos da Base Station
 - Economiza energia
 - Reduz interferências
2. **Gateway e Base Stations (média/longa distância):** atpl 150

- Alta potência para alcançar múltiplos nós
- Gateway precisa comunicar com todas as BS
- BS precisam retransmitir dados dos sensores

3. Aglomerados Ambientais: atpl 150

- Distância de 15m até BS (maior que sensores de estacionamento)
- Distância de 18m entre aglomerados
- Necessitam maior alcance

Adaptação dinâmica à distância (conceito):

Para adaptar automaticamente o PL à distância, poderia implementar-se:

```
# Exemplo conceitual de ajuste dinâmico
set distancia 15      # metros
set pl_base 100
set pl_ajustado pl_base + (distancia * 3)
atpl pl_ajustado
```

No nosso projeto:

- Sensores estacionamento (3-8m): PL = 20
- Sensores ambientais (15-18m): PL = 150
- Gateway/BS (variável): PL = 150

Regra prática:

- Distâncias curtas (<10m): PL 20-50
- Distâncias médias (10-20m): PL 100-150
- Distâncias longas (>20m): PL 150-200

1.4. Outros Comandos Importantes

Envio de Mensagens

Comando: send

Sintaxe: send <mensagem> <destino_id>

Exemplos:

```
send p BS_ID          # Envia pacote para Base Station
send "VENT_ON" VENT_ID # Envia comando para ventilador
```

Receção de Mensagens

Comando: receive

Sintaxe:

receive <variavel> <timeout_ms>

Exemplos:

receive msg 1000 # Gateway aguarda 1s

receive msg_downlink sample_period # Aglomerado aguarda período configurável

Marcação Visual

Comando: mark

Sintaxe: mark <valor>

Exemplo: mark s # Atualiza LED conforme estado do sensor (0=verde, 1=vermelho)

Usado em: Sensores de estacionamento para feedback visual

Temporização

Comando: time

Sintaxe: time <variavel>

Exemplo:

time timestamp

data p timestamp frame my_id GW_ID s

Função: Obtém o timestamp atual da simulação

Espera/Delay

Comando: delay

Sintaxe: delay <milissegundos>

Exemplos:

randb backoff 0 250 # Gera delay aleatório

delay backoff # Espera tempo aleatório (evita colisões)

delay 1000 # Espera 1 segundo fixo

Impressão na Consola

Comando: cprint

Sintaxe: cprint <texto> <variavel1> <texto2> <variavel2> ...

Exemplo: *cprint "Sensor" my_id "ENVIOU (Park):" p*

Leitura de Dados Remotos (de ficheiros .evt)

Comando: *rdata*

Sintaxe: *rdata <mensagem> <var1> <var2> ...*

Exemplo: *rdata msg_downlink cmd_tipo cmd_valor*

Função: Extrai valores de uma string formatada (alternativa a *vdata/vget*)

Ciclo Infinito

Comando: *loop ... end*

Sintaxe:

```
loop
  # Código que se repete infinitamente
end
```

Usado em: Todos os scripts do projeto (funcionamento contínuo)

2. ARQUITETURA DO SISTEMA

2.1. Visão Geral

O sistema implementa uma **infraestrutura IoT para Smart Parking e**

Monitorização Ambiental no campus de Azurém com:

- **20 sensores ultrassónicos** (estacionamento)
- **2 aglomerados de sensores ambientais** (CO2, PM10, temperatura, humidade)
- **2 ventiladores** (atuadores)
- **4 Base Stations** (routers multi-hop)
- **1 Gateway** (agregador e conversor de protocolos)

Tecnologias:

- **Zigbee** → Sistema de estacionamento (curtas distâncias)
- **LoRa** → Sistema ambiental (médias distâncias)
- **IP/Ethernet** → Gateway para rede GNS3

2.2. Componentes e Funções

Gateway (nó 30)

- **Ficheiro:** Gateway.csc
- **Funções:**
 - o Recebe e processa TODAS as mensagens da rede
 - o Distingue mensagens PARK (estacionamento) e DATA (ambiental)
 - o Imprime estado dos lugares de estacionamento
 - o Imprime leituras ambientais formatadas
 - o **Capacidade de downlink:** Envia comandos CONFIG para alterar período de amostragem
 - o Regista mensagens desconhecidas para análise
- **Potência:** atpl 150 (alta, comunicação com múltiplos nós)
- **Destinos downlink:**
 - o Aglomerado_1 (ID 27) via BS
 - o Aglomerado_2 (ID 33) diretamente

Sensores de Estacionamento (4 filas × 5 sensores = 20 sensores)

Ficheiros: Sensores_Fila1.csc, Sensores_Fila2.csc, Sensores_Fila3.csc, Sensores_Fila4.csc

IDs dos sensores:

- Fila 1: nós 21, 18, 19, 20, 22
- Fila 2: nós 10, 11, 12, 13, 14
- Fila 3: nós 31, 2, 34, 35, 36
- Fila 4: nós 37, 38, 39, 40, 41

Funções:

- Leem sensor ultrassónico digital (dreadsensor)
- Detetam mudança de estado (livre ↔ ocupado)
- Envia mensagem PARK apenas quando há mudança
- Atualizam marcador visual (LED: verde=livre, vermelho=ocupado)
- **Backoff aleatório** (0-250ms) para evitar colisões

Potência: atpl 20 (baixa, sensores próximos das BS)

Destinos:

- Fila 1 → BS_Origem_1 (ID 23)
- Fila 2 → BS_Recetor_1 (ID 24)
- Fila 3 → BS_Origem_2 (ID 25)
- Fila 4 → BS_Recetor_2 (ID 26)

Distâncias:

- Entre sensores: 3 metros
- Sensores → BS: 8 metros

Aglomerados de Sensores Ambientais

Ficheiros: Aglomerado_1.csc (nó 27), Aglomerado_2.csc (nó 33)

Sensores integrados em cada aglomerado:

- Temperatura
- Humidade
- CO2
- PM10

Funções:

- Leem sensores analógicos (areadsensor) de ficheiros .evt
- Envia pacote DATA periodicamente (10 segundos por defeito)
- **Inteligência local:** Controlam ventilador autonomamente
 - o Se $CO_2 > 1000$ ppm → enviam "VENT_ON"
 - o Se $CO_2 \leq 1000$ ppm → enviam "VENT_OFF"
- **Recebem comandos downlink CONFIG** do Gateway
- Ajustam período de amostragem dinamicamente
- **Backoff aleatório** (0-250ms)

Potência: atpl 150 (alta, distâncias maiores)

Destinos:

- Aglomerado_1 → BS_Origem_1 (ID 23) → Gateway (ID 30)
- Aglomerado_2 → Gateway (ID 30) diretamente

Distâncias:

- Entre aglomerados: 18 metros
- Aglomerado → BS: 15 metros

Ventiladores (nós 67 e 69)

Ficheiro: Ventilador.csc

Funções:

- Recebem comandos "VENT_ON" / "VENT_OFF"
- Atuam conforme comando recebido
- Atualizam marcador visual (simulação liga/desliga)

Controlo:

- Ventilador 67 ← Aglomerado_1 (nó 27)
- Ventilador 69 ← Aglomerado_2 (nó 33)

Base Stations (4 nós: 23, 24, 25, 26)

Ficheiros: BS_Origem_1.csc, BS_Recetor_1.csc, BS_Origem_2.csc, BS_Recetor_2.csc

Funções:

- Funcionam como **routers multi-hop**
- **NÃO processam dados** (apenas retransmitem)
- Recebem mensagens dos sensores
- Reencaminham para o Gateway (ID 30)
- Criam topologia mesh para aumentar cobertura

Potência: atpl 150 (retransmissão)

Topologia:

- BS_Origem_1 (23) → recebe de Fila 1 e Aglomerado_1 → envia para Gateway
- BS_Recetor_1 (24) → recebe de Fila 2 → envia para Gateway
- BS_Origem_2 (25) → recebe de Fila 3 → envia para Gateway
- BS_Recetor_2 (26) → recebe de Fila 4 → envia para Gateway

2.3. Fluxo de Dados

Sistema de Estacionamento (UPLINK)

Sensor Fila 1 (nó 21)

↓ [PARK packet] PL=20

BS_Origem_1 (nó 23)

↓ [retransmite] PL=150

Gateway (nó 30)

↓ [processa e imprime]
Console: "ESTADO LUGAR 21: OCUPADO"

Sistema Ambiental (UPLINK)

Aglomerado_1 (nó 27)
↓ [DATA packet] PL=150
BS_Origem_1 (nó 23)
↓ [retransmite] PL=150
Gateway (nó 30)
↓ [processa e imprime]
Console: "DADOS AMBIENTAIS | Temp: 22°C | CO2: 850 ppm ..."

Controlo de Ventilação (LOCAL)

Aglomerado_1 (nó 27)
↓ [lê CO2 > 1000 ppm]
↓ [decisão local]
↓ [VENT_ON]
Ventilador (nó 67)
↓ [liga]

Importante: Esta decisão é local, não depende do Gateway!

Configuração Remota (DOWNLINK)

Gateway (nó 30)
↓ [CONFIG packet: novo período = 15000ms]
BS_Origem_1 (nó 23)
↓ [retransmite]
Aglomerado_1 (nó 27)
↓ [recebe e atualiza sample_period]
Console: "Sensor 27 RECEBEU DOWNLINK (Config): 15000 ms"

3. DECISÕES DE DESIGN E JUSTIFICAÇÕES

3.1. Sensores Direcionais (TESTE EXPERIMENTAL)

O que foi testado:

- Substituição de sensores ultrassónicos standard por **sensores direcionais** numa das filas (teste)

Resultados observados:

- **Resultado positivo:** Redução de falsas deteções
- Melhor precisão na deteção de ocupação específica do lugar
- Menor interferência entre sensores adjacentes (separados apenas 3m)

Vantagens dos sensores direcionais:

- Foco no espaço específico do lugar de estacionamento
- Reduzem ruído de objetos laterais
- Melhor desempenho em espaços apertados

Conclusão:

- Teste bem-sucedido, viável para implementação em cenário real
- Recomendável para parques com lugares muito próximos

3.2. Inteligência Local vs. Centralizada (DECISÃO CRÍTICA)

Porquê processar no Aglomerado e não no Gateway?

Decisão: Implementar **controlo do ventilador localmente no aglomerado** (nó 27/33), não no Gateway (nó 30)

Razões técnicas:

1. Autonomia operacional:

- o Sistema continua a funcionar mesmo com perda de ligação ao Gateway
- o Ventilação crítica para segurança (evitar acumulação CO2)
- o Não depende de comunicação externa

2. Redução de latência:

- o Decisão instantânea (ms) vs. round-trip pela rede (segundos)
- o Aglomerado → Ventilador: 1 hop
- o Aglomerado → Gateway → Aglomerado → Ventilador: 4 hops

3. Redução de tráfego na rede:

- o Comandos de ventilação não passam pelo Gateway
- o Menos carga nas Base Stations
- o Mais largura de banda para dados de monitorização

4. Resiliência a falhas:

- o Se Gateway falha: ventilação continua operacional
- o Se rede congestionada: atuação não é afetada
- o Sistema crítico isolado de sistema de monitorização

5. Eficiência energética:

- o Menos mensagens transmitidas
- o Menos retransmissões
- o Menor consumo nos aglomerados

Exemplo de cenário crítico:

Situação: Ligação Gateway ↔ Base Station falha

Com inteligência CENTRALIZADA (Gateway):

- ✗ CO2 sobe para 1500 ppm
- ✗ Gateway não recebe dados
- ✗ Comando de ventilação não é enviado
- ✗ PERIGO!

Com inteligência LOCAL (Aglomerado):

- ✓ CO2 sobe para 1500 ppm
- ✓ Aglomerado detecta localmente
- ✓ Aglomerado envia "VENT_ON" diretamente
- ✓ Ventilador liga
- ✓ Sistema seguro!

Trade-off:

- Gateway perde controle direto sobre ventilação
- Mas ganha robustez e segurança do sistema

3.3. Arquitetura Multi-Hop

Porquê usar Base Stations?

- Parque subterrâneo: obstáculos físicos (paredes, pilares)
- Distâncias superiores ao alcance direto (20+ metros)
- Criar topologia mesh para redundância

Benefícios:

- Cobertura completa do parque
- Caminhos alternativos em caso de falha
- Escalabilidade (fácil adicionar mais sensores)

4. FORMATOS DE MENSAGENS (PROTOCOLO)

4.1. Mensagem PARK (Estacionamento)

Formato: data p timestamp "PARK" sensor_id gateway_id status

Campos:

- timestamp: momento da detecção
- "PARK": identificador do tipo

- sensor_id: ID do sensor (21, 18, 19, ...)
- gateway_id: destino (30)
- status: 0 (livre) ou 1 (ocupado)

Exemplo real:

[12:34:56] PARK 21 30 1

Significado: No tempo 12:34:56, sensor 21 reportou lugar ocupado (1)

4.2. Mensagem DATA (Ambiental)

Formato:

data p timestamp "DATA" aglomerado_id gateway_id temp hum co2 pm10

Campos:

- timestamp: momento da leitura
- "DATA": identificador do tipo
- aglomerado_id: ID do aglomerado (27 ou 33)
- gateway_id: destino (30)
- temp: temperatura (°C)
- hum: humidade (%)
- co2: CO2 (ppm)
- pm10: partículas PM10

Exemplo real:

[12:34:56] DATA 27 30 22.5 65 850 45

Significado: Aglomerado 27 reportou: 22.5°C, 65% hum, 850ppm CO2, 45 PM10

4.3. Mensagem CONFIG (Downlink)

Formato:

data p_config timestamp "CONFIG" gateway_id novo_periodo

Campos:

- timestamp: momento do envio
- "CONFIG": identificador do tipo
- gateway_id: origem (30)
- novo_periodo: novo período de amostragem (ms)

Exemplo real:

[12:35:00] CONFIG 30 15000

Significado: Gateway ordena novo período de 15 segundos

4.4. Comandos de Ventilação (Local)

Formato: String simples

Comandos:

- "VENT_ON" → Liga ventilador
- "VENT_OFF" → Desliga ventilador

Enviados por: Aglomerado_1 (27) → Ventilador (67), Aglomerado_2 (33) → Ventilador (69)

5. ESTATÍSTICAS DA SIMULAÇÃO

Da imagem anexa (tempo 0.1228s):

- **Mensagens SENT:** 2.0 B2.0 (Bytes)
- **Mensagens RECEIVED:** 0.0 (0.0 Bytes)
- **Mensagens LOST:** 0.0 (0.0 Bytes)
- **Mensagens ACK:** 0.0 (0.0 Bytes)

Interpretação:

- Início da simulação
- 2 mensagens já enviadas
- Nenhuma ainda recebida/perdida (propagação em curso)
- Simulação a funcionar corretamente

6. PERGUNTAS POSSÍVEIS DO PROFESSOR E RESPOSTAS

Perguntas sobre Comandos

P1: "Como é que se cria uma trama em SenScript?" R: Com o comando data. Por exemplo: data p timestamp "PARK" my_id GW_ID status. Este comando cria uma mensagem estruturada com múltiplos campos que depois é enviada com send p destino_id.

P2: "E como se extraem os campos dessa trama?" R: Primeiro converte-se a mensagem recebida num vetor com `vdata v msg`, depois extrai-se cada campo com `vget` variavel `v` indice. Por exemplo, no Gateway fazemos `vget tipo v 1` para extrair o tipo de mensagem (PARK ou DATA).

P3: "Como obter o ID de um nó?" R: Com `atget id` variavel. Por exemplo, `atget id my_id` guarda o ID do nó na variável `my_id`. Usamos isto em todos os scripts para identificar o nó automaticamente.

P4: "Explique como alterar a potência de transmissão." R: Com o comando `atpl` valor. Usamos `atpl 20` nos sensores de estacionamento porque estão perto das Base Stations (3-8m), e `atpl 150` no Gateway e aglomerados que precisam de maior alcance (15-18m). A potência deve ser ajustada conforme a distância para economizar energia e reduzir interferências.

Perguntas sobre Arquitetura

P5: "Qual é a função das Base Stations?" R: Funcionam como routers multi-hop. Recebem mensagens dos sensores e reencaminham para o Gateway. Não processam os dados, apenas retransmitem. Servem para estender a cobertura da rede em espaços grandes ou com obstáculos.

P6: "Porque é que não colocaram toda a lógica no Gateway?" R: Por autonomia e resiliência. O controlo dos ventiladores é feito localmente nos aglomerados. Se houver perda de ligação com o Gateway (por exemplo, falha de rede ou congestionamento), o sistema de ventilação continua a funcionar. Isto é crítico para segurança, porque CO2 elevado é perigoso. Além disso, reduz latência e tráfego na rede.

P7: "Como funciona o sistema de downlink?" R: O Gateway envia comandos CONFIG através das Base Stations para os aglomerados. O comando contém um novo período de amostragem. Os aglomerados recebem estes comandos no final do ciclo de espera (`receive msg_downlink sample_period`) e atualizam o período dinamicamente, permitindo configuração remota durante operação.

Perguntas sobre Decisões de Design

P8: "Falem sobre os sensores direcionais." R: Testámos sensores direcionais numa das filas em vez dos ultrassónicos standard. O resultado foi positivo: reduziu falsas

deteções e aumentou precisão, especialmente importante porque os lugares estão separados apenas 3 metros. Os sensores direcionais focam-se no espaço específico do lugar, ignorando objetos laterais. É uma solução viável para implementação real em parques com lugares muito próximos.

P9: "Porque implementaram inteligência local nos aglomerados?" R: Três razões principais:

1. **Segurança:** O sistema continua a funcionar mesmo com perda de ligação ao exterior.
2. **Latência:** Decisão instantânea (ms) versus esperar round-trip pela rede (segundos).
3. **Eficiência:** Menos tráfego na rede, comandos de ventilação não congestionam as Base Stations.

Se o Gateway falhar ou a rede congestionar, a ventilação - que é crítica para segurança - continua operacional.

Perguntas sobre Implementação

P10: "Como evitam colisões na transmissão?" R: Usamos backoff aleatório. Antes de cada transmissão, geramos um delay aleatório entre 0 e 250ms com `randb backoff 0 250` e esperamos com `delay backoff`. Isto distribui as transmissões no tempo e reduz colisões, especialmente quando múltiplos sensores detetam mudanças simultaneamente.

P11: "Como distinguem mensagens PARK de DATA no Gateway?" R: Primeiro recebemos a mensagem e convertemos em vetor: `vdata v msg`. Depois extraímos o campo `tipo: vget tipo v 1`. Se `tipo == "PARK"`, processamos dados de estacionamento. Se `tipo == "DATA"`, processamos dados ambientais. Cada tipo tem campos diferentes nas posições seguintes do vetor.

P12: "Os sensores enviam dados continuamente?" R: Não. Os sensores de estacionamento só enviam quando há mudança de estado (livre ↔ ocupado). Os aglomerados ambientais enviam periodicamente (10 segundos por defeito, configurável via `downlink`). Isto economiza energia e largura de banda.

Perguntas sobre Funcionalidades

P13: "Como funciona o controlo automático dos ventiladores?" R: Os aglomerados leem CO2 localmente. Se $CO_2 > 1000$ ppm, enviam "VENT_ON" diretamente para o ventilador. Se CO2 desce abaixo de 1000 ppm, enviam "VENT_OFF". Esta decisão é totalmente local e autónoma, não depende do Gateway.

P14: "Qual é o período de amostragem dos sensores ambientais?" R: Por defeito 10 segundos (10000ms), mas é configurável remotamente. O Gateway pode enviar comando CONFIG com novo período, e os aglomerados atualizam dinamicamente a variável sample_period.

P15: "Como visualizam o estado dos lugares de estacionamento?" R: Usamos o comando mark para atualizar marcadores visuais no CupCarbon. Quando o sensor deteta estado 0 (livre), o marcador fica verde. Estado 1 (ocupado), fica vermelho. Isto permite visualização imediata na simulação.

Perguntas sobre Protocolos

P16: "Que tecnologias de comunicação sem fios usaram?" R: Zigbee para os sensores de estacionamento (curtas distâncias, 3-8m) e LoRa para os sensores ambientais (médias distâncias, 15-18m). O Gateway depois converte para IP/Ethernet para integração com a rede GNS3 da Fase A.

P17: "Expliquem o formato das mensagens PARK." R: data p timestamp "PARK" sensor_id gateway_id status. Os campos são: timestamp (momento), tipo ("PARK"), ID do sensor, ID do Gateway destino, e status (0=livre, 1=ocupado). Por exemplo: "12:34:56 PARK 21 30 1" significa sensor 21 reportou ocupado.

P18: "E das mensagens DATA?" R: data p timestamp "DATA" aglomerado_id gateway_id temp hum co2 pm10. Inclui temperatura, humidade, CO2 e PM10. Por exemplo: "12:34:56 DATA 27 30 22.5 65 850 45" - aglomerado 27 reportou 22.5°C, 65% humidade, 850ppm CO2, 45 PM10.

Perguntas sobre Escalabilidade

P19: "Como adicionariam mais sensores à rede?" R: Basta criar o nó no CupCarbon, configurar o ID, potência e destino (Base Station mais próxima). O código é reutilizável. Para sensores de estacionamento, duplicar Sensores_FilaX.csc e ajustar

BS_ID. Para ambientais, duplicar Aglomerado_X.csc. Não é necessário reconfigurar Gateway ou outras Base Stations.

P20: "E se quisessem adicionar outro tipo de sensor?" R: Criar novo tipo de mensagem (ex: "TEMP" para temperatura isolada), enviar para o Gateway, e adicionar novo bloco de processamento no Gateway.csc:

```
else
  if (tipo == "TEMP")
    vget temp v 2
    cprint "Temperatura:" temp
  end
end
```

Perguntas sobre Testes e Validação

P21: "Como testaram o sistema?" R: Simulação no CupCarbon com múltiplos cenários:

1. Detecção de ocupação/libertação de lugares
2. Leituras ambientais periódicas
3. Ativação automática de ventiladores ($\text{CO}_2 > 1000\text{ppm}$)
4. Recepção de comandos downlink
5. Funcionamento multi-hop através das Base Stations

P22: "E se uma Base Station falhar?" R: Depende da topologia. Se houver caminho alternativo (mesh), mensagens são reencaminhadas. Se não, sensores dessa zona ficam isolados. Numa implementação real, implementaríamos roteamento dinâmico e redundância de BS.

7. FICHEIROS DO PROJETO

Ficheiros de Código Fonte (.csc)

1. Gateway.csc
2. Sensores_Fila1.csc
3. Sensores_Fila2.csc
4. Sensores_Fila3.csc
5. Sensores_Fila4.csc
6. Aglomerado_1.csc

7. Aglomerado_2.csc
8. Ventilador.csc
9. BS_Origem_1.csc
10. BS_Origem_2.csc
11. BS_Recetor_1.csc
12. BS_Recetor_2.csc

Ficheiros de Amostras de Sensores (.evt)

1. co2.evt
2. co22.evt
3. humidade.evt
4. humidade2.evt
5. pm10.evt
6. pm102.evt
7. temperatura.evt
8. temperatura2.evt

8. REFERÊNCIAS RÁPIDAS

IDs dos Nós

- Gateway: 30
- Sensores Fila 1: 21, 18, 19, 20, 22
- Sensores Fila 2: 10, 11, 12, 13, 14
- Sensores Fila 3: 31, 2, 34, 35, 36
- Sensores Fila 4: 37, 38, 39, 40, 41
- Aglomerado_1: 27
- Aglomerado_2: 33
- Ventilador_1: 67
- Ventilador_2: 69
- BS_Origem_1: 23
- BS_Recetor_1: 24
- BS_Origem_2: 25
- BS_Recetor_2: 26

Potências de Transmissão

- Sensores estacionamento: 20
- Gateway, BS, Aglomerados: 150

Distâncias

- Entre sensores estacionamento: 3m
- Sensores → Base Station: 8m
- Entre aglomerados: 18m
- Aglomerados → Base Station: 15m

Períodos e Tempos

- Amostragem ambiental: 10s (10000ms) - configurável
- Backoff aleatório: 0-250ms
- Delay sensores estacionamento: 1s
- Timeout recepção Gateway: 1s

9. DICAS PARA A APRESENTAÇÃO

Pontos Fortes a Destacar

1. Sistema funcional completo (20 sensores + 2 aglomerados + atuadores)
 2. Arquitetura multi-hop robusta
 3. Inteligência local (autonomia)
 4. Configuração remota (downlink)
 5. Teste inovador (sensores direcionais)
 6. Protocolo de comunicação bem definido
 7. Backoff para evitar colisões
 8. Eficiência energética (transmissões sob demanda)
- Conhecer BEM os comandos principais: data, vget, atget, atpl
 - Saber explicar PORQUÊ inteligência local
 - Ter exemplos de mensagens PARK e DATA decorados
 - Conhecer os IDs principais (Gateway 30, Aglomerados 27 e 33)